Apr 27

Streamlit vs Gradio in 2025: Comparing AI-App Frameworks

Naoto Shibata Co-founder, CEO

Streamlit / Gradio as Frameworks for AI Apps

Streamlit and Gradio are both popular for letting you build interactive web apps using nothing but Python.

Although they share the "light-weight Python code → web app" philosophy, their focus differs sharply:

- **Streamlit = strong at customization and complex UI/UX**
- **Gradio = strong at publishing ML models and running long-lasting inference jobs**

Both require setting up a Python environment and writing code—but what if you could skip that entirely? We'll explore that option later.

Below we examine each framework's strengths and use-cases when you're building and operating AI apps inside an organization.

Streamlit Use-Cases — Combining "Presentation" and "Operational Robustness"

Streamlit shines where you need the ease of building interactive web apps **and** the flexibility to handle feature requests that crop up once the tool is in production.

UI Customization

In addition to built-in widgets like `st.line_chart` and `st.button`, there's a rich ecosystem of community components such as **streamlit-extras** and **st-chat**.

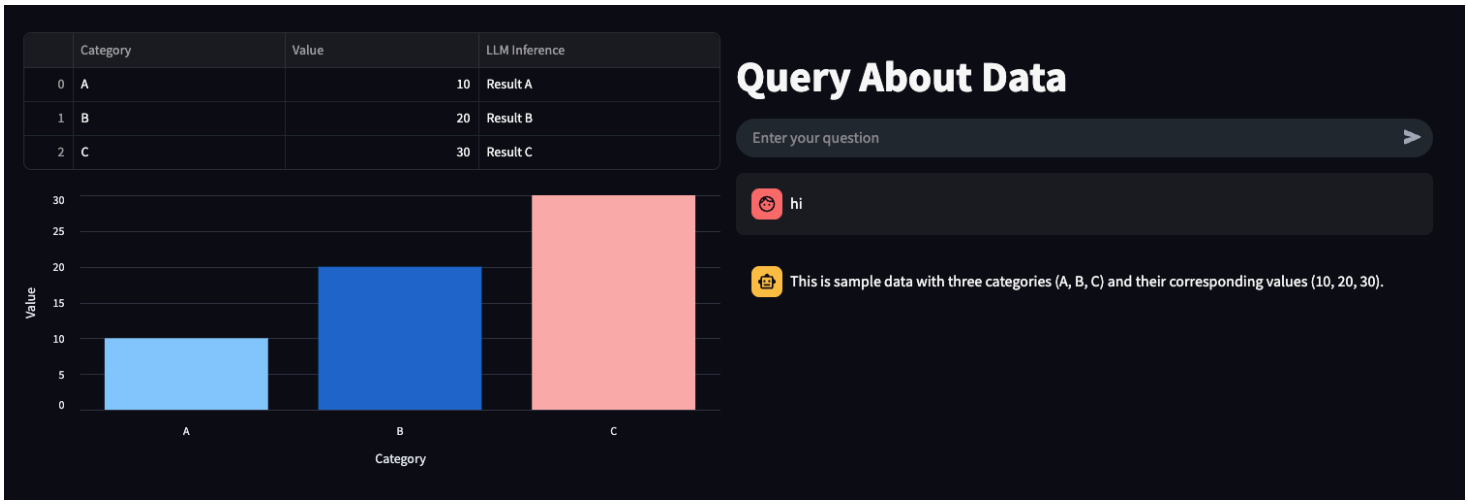
Because you can inject CSS or leverage the theme API, you can freely match corporate brand colors and layouts.

```
import streamlit as st

# Create a button
if st.button('Click me'):
    st.write('Button clicked!')

# Customize layout and theme using CSS
st.markdown(
    """
    <style>
    .stButton>button {
        background-color: #4CAF50;
        color: white;
    }
    </style>
    """,
    unsafe_allow_html=True
)
```

Hybrid Data + AI Apps

By layering embeddings or LLM outputs on top of `st.dataframe` or `st.altair_chart`, you can naturally implement compound UIs such as *"a chat app with charts."* Streamlit originally targeted data scientists, so it supports many visualization libraries and excels at presenting data.

Components for real-time AI interaction—for example `st.chat_message` for streaming chat updates—have also matured, making it easy to build hybrid apps that mix data and AI.




```
import streamlit as st
import pandas as pd
import altair as alt

st.set_page_config(layout="wide")
col1, col2 = st.columns([1, 1])

data = pd.DataFrame({
    'Category': ['A', 'B', 'C'],
    'Value': [10, 20, 30],
    'LLM Inference': ['Result A', 'Result B', 'Result C']
})

with col1:
```

```
    # Display the data
    st.dataframe(data, use_container_width=True)

    # Bar chart
    chart = alt.Chart(data).mark_bar(color='#4A90E2').encode(
        x=alt.X('Category', axis=alt.Axis(labelAngle=0)),
        y=alt.Y('Value', title='Value'),
        tooltip=['Category', 'Value', 'LLM Inference'],
        color=alt.Color('Category', legend=None)
    ).properties(height=300)
    st.altair_chart(chart, use_container_width=True)

with col2:
    st.title("Query About Data")
    if "messages" not in st.session_state:
        st.session_state.messages = []
    for m in st.session_state.messages:
        with st.chat_message(m["role"]):
            st.markdown(m["content"])

    prompt = st.chat_input("Enter your question")
    if prompt:
        with st.chat_message("user"):
            st.markdown(prompt)
        st.session_state.messages.append({"role": "user", "content": prompt})

        with st.spinner("Getting response from OpenAI…"):
            # call OpenAI API
            with st.chat_message("assistant"):
                answer = "This sample dataset has three categories (A, B, C) with values 10, 20, 30."
                st.markdown(answer)
            st.session_state.messages.append({"role": "assistant", "content": answer})
```

Reliability in Production

Because Streamlit runs each user session as an isolated script, you don't worry about global-state clashes and side effects, which lowers debugging overhead.

Its weakness is managing *very* long-running ML inference or high concurrent loads; you typically off-load heavy jobs to a separate API (FastAPI, etc.) and have Streamlit poll or use WebSockets for progress updates.

Use-Cases Best Suited to Streamlit

- Internal dashboards + AI assistants
- Operational tools with complex forms or dynamic reporting
- Proof-of-concepts that should move straight to branded production

---

Gradio Use-Cases — Strong at Publishing AI/ML Models & Long-Running Inference

Gradio offers less UI flexibility than Streamlit but focuses on **making ML models easy to share**. Its simple, highly [illegible] ABI ships only the components you really need.

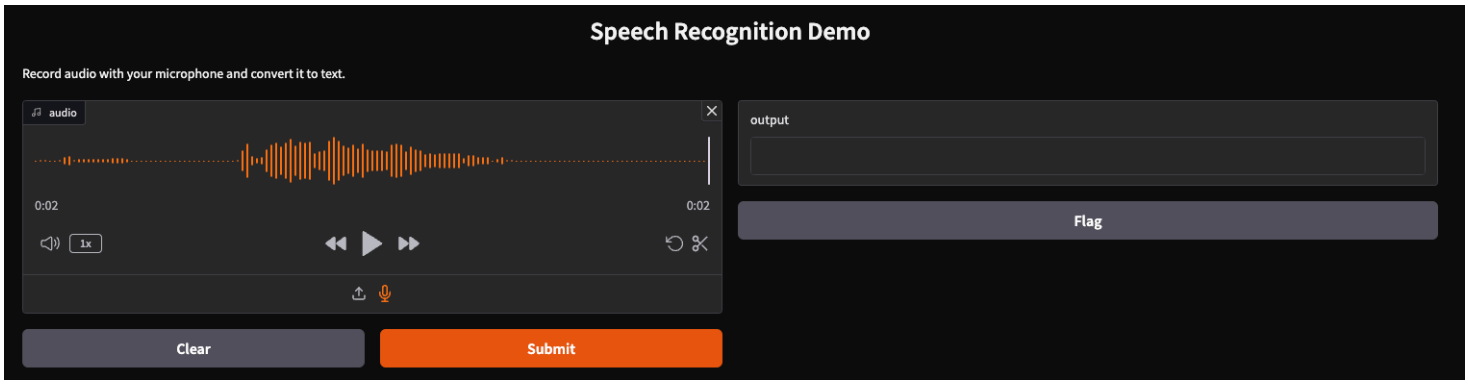
Squadbase

Key advantages include ready-made widgets for tasks Streamlit lacks out of the box (image generation, speech recognition, etc.) and built-in helpers that simplify inference job management.

Components Specialized for Model I/O

Widgets like `gr.Audio`, `gr.Image`, and video streams turn multimedia I/O into one-liners—perfect for showing trained models.

Even a voice-recording app that transcribes audio takes only a few lines:




```
import gradio as gr

def transcribe_audio(audio):
    return "Audio has been recorded. In a real application, speech recognition would be performed here."

demo = gr.Interface(
    fn=transcribe_audio,
    inputs=gr.Audio(type="filepath"),
    outputs="text",
    title="Speech Recognition Demo",
    description="Record audio with your microphone and convert it to text."
)

if __name__ == "__main__":
    demo.launch()
```

`gr.Flagging` automatically stores user feedback under `.gradio/flagged`, giving you a zero-code feedback loop for continual learning.

Smart Execution of "Long Inference" with Queues

Gradio ships with an async backend queue that scales to thousands of concurrent users. Components like progress bars keep UX smooth even for tasks that take tens of seconds or minutes (image generation, speech-to-speech, etc.).

The sample below caps concurrency at 2 (ideal when you have only two GPUs) and uses `gr.Progress` to show real-time status:

```python
import gradio as gr
import time

def image_gen(prompt, progress=gr.Progress()):
    progress(0, desc="Starting")
    time.sleep(1)
    progress(0.5)
    time.sleep(1)
    progress(1)
    return "https://www.gradio.app/_app/immutable/assets/gradio.CHB5adID.svg"

with gr.Blocks() as demo:
    prompt = gr.Textbox()
    image = gr.Image()
    btn1 = gr.Button("Generate Image via model 1")
    btn2 = gr.Button("Generate Image via model 2")
    btn3 = gr.Button("Generate Image via model 3")
    btn1.click(image_gen, prompt, image, concurrency_limit=2, concurrency_id="gpu_queue")
    btn2.click(image_gen, prompt, image, concurrency_id="gpu_queue")
    btn3.click(image_gen, prompt, image, concurrency_id="gpu_queue")

demo.launch()
```

Instant Sharing & Instant API Generation

Set `demo.launch(share=True)` and your Gradio app is hosted on Hugging Face with one line. No cloud setup—just tweak your local code and share the link.

Hosting on Hugging Face Spaces or your own server **also auto-generates REST/WS APIs** (curl, Python, JavaScript) whose I/O mirrors the demo UI.

While the UI is less customizable, the framework automates every step of demo distribution. Note, though, that public share links expire in a week and rely on Gradio/Hugging Face infra, so long-term internal apps with sensitive data require extra authentication and access-control work.

Use-Cases Best Suited to Gradio

- Rapid publication of research results or model demos
- AI apps for image generation / speech recognition / TTS or other long-running inference

---

Common Challenges: What Both Frameworks Share

Both Streamlit and Gradio have made building AI apps significantly easier than traditional web development. However, they share some common requirements that can become barriers—especially for teams that just want to visualize data.

- Python installation and version management
- Virtual environments (venv, conda, poetry)
- Dependency management and version conflicts
- IDE or code editor setup

For teams with mixed technical backgrounds, this initial hurdle can slow adoption considerably.

Deployment Complexity

Building the app is just step one. To share it with your team, you need to:

- Choose and configure a hosting platform (AWS, GCP, Heroku, etc.)
- Set up CI/CD pipelines for updates
- Manage environment variables and secrets securely
- Handle SSL certificates and custom domains

Authentication and Access Control

Neither framework includes built-in authentication. Adding secure access typically means:

- Integrating with an identity provider (Auth0, Okta, Google, etc.)
- Implementing session management
- Managing user roles and permissions
- Handling SSO requirements for enterprise environments

The Real Question

If your primary goal is **data visualization and dashboards**—not building custom AI model interfaces—do you really need to set up a Python environment, write code, and manage deployment?

Beyond Python Frameworks: The Vibe Coding Approach

What if you could build data dashboards without any of these hurdles?

Squadbase takes a fundamentally different approach. Instead of coding in Python, you describe what you want in natural language and the AI builds it for you—entirely in the browser.

No Environment Setup Required

Squadbase runs entirely in your browser. No Python installation, no virtual environments, no dependency conflicts. If you can open a web browser, you can build a dashboard.

Squadbase

This means business analysts, operations teams, and anyone who works with data can create dashboards without waiting for engineering support.

Powerful Data Connections

Connect to virtually any data source.

- **Files**: Excel, CSV, Jupyter Notebooks
- **Databases**: PostgreSQL, Snowflake, BigQuery, MySQL
- **SaaS APIs**: Salesforce, Notion, HubSpot, and more

Squadbase automatically understands your data schema and sets up secure connections in seconds—no manual configuration required.

Real Code, Not Black Box

Unlike no-code tools that lock you into proprietary systems, Squadbase generates **real Next.js code**. This matters because:

- You can see and edit every line of code the AI writes
- Export your project and customize further if needed
- Version control via GitHub integration
- Extend with any JavaScript/TypeScript library

You get the speed of no-code with the flexibility of code.

Human-in-the-Loop: Stay in Control

Squadbase keeps you in the loop. You review and refine AI-generated queries and visualizations—no surprises, full accuracy. This transparency is what separates Squadbase from "black-box" AI dashboard tools.

When something doesn't look right, you can see exactly what the AI did and correct it through conversation.

Built-in Authentication and Sharing

No need to implement authentication yourself. Squadbase includes

- Role-based access control out of the box
- SSO integration for enterprise teams
- **$0 for viewers**—share with your entire team without per-seat fees

When Squadbase Excels

Squadbase is particularly powerful when

- Your goal is **data visualization and dashboards** (BI use cases)

Squadbase

- Your team includes **non-engineers** who need to build or modify dashboards
- You want to **skip the DevOps** and focus on insights
- You need **production-ready sharing** from day one

Conclusion: Choosing Your Path

Beyond their UI components, Streamlit, Gradio, and Squadbase diverge in how they help teams work with data. Understanding each tool's strengths is critical.

When to Choose Streamlit

- You're building **hybrid data + AI applications** that go beyond simple dashboards
- Your team is **comfortable with Python** and environment setup
- You need **extensive UI customization** and are willing to invest in CSS
- You prefer the **larger ecosystem** and active community support

When to Choose Gradio

- Your primary use case is **ML model demos** with specialized input/output interfaces
- You need **built-in queue systems** for long-running inference tasks
- **Instant sharing via Hugging Face** aligns with your workflow
- You're building **model-centric applications** rather than data dashboards

When to Choose Squadbase

- Your goal is **BI dashboards and data visualization**
- You want to **skip environment setup** entirely
- Your team includes **non-engineers** who need to build dashboards
- You need **built-in authentication and sharing** without DevOps overhead
- You value **transparent, editable code** over black-box solutions

Comparison at a Glance

Feature	Streamlit	Gradio	Squadbase
Primary Use Case	Data + AI apps	ML model demos	BI dashboards
Interface	Python code	Python code	Natural language

Feature	Streamlit	Gradio	Squadbase
Environment Setup	Required	Required	None (browser-only)
Deployment	Self-managed	Self-managed or HF	Built-in
Authentication	DIY	DIY	Built-in
Code Transparency	Full (Python)	Full (Python)	Full (Next.js)
Non-engineer Friendly	Limited	Limited	Yes
Cost for Viewers	Hosting-dependent	Hosting-dependent	$0

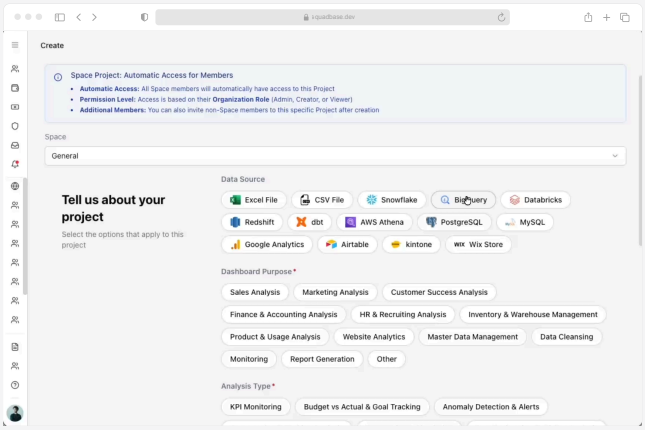
Ready to try the vibe coding approach?

If your goal is building data dashboards and you want to skip the Python setup, try Squadbase free. Go from spreadsheet to shareable dashboard in minutes—no terminal required.

Get your first dashboard in 10 minutes

Free to get started · No credit card required

Start talking to your data



Squadbase